
Running Large Language Models on Everyday Devices: A Survey of Inference Optimizations

Bhargav Chirumamilla*
Independent Researcher
REPLACE_WITH_EMAIL

Abstract

Most people who will ever use a large language model (LLM) will not sit in front of a datacenter GPU. They will sit in front of a laptop, a phone, or a tablet that already runs a browser, an editor, and an operating system—and they will expect the model to share that machine. This survey treats *everyday-device inference* as the primary setting, not as a footnote to cloud serving. We organize the literature into a bottleneck-first taxonomy spanning ten families: weight and activation compression; key–value (KV) cache and context memory; attention and prefill kernels; speculative and parallel decoding; batching and scheduling; model and expert parallelism; architecture and model-size choices; adapters; compilation and runtimes; and application-level controls. For each family we state the governing equations, map systems papers onto the device constraint (shared DRAM, thermals, a single interactive user, background apps), and contrast what transfers from datacenter stacks with what does not. We then report a controlled empirical study on Apple Silicon unified memory—a Mac M3 with 24 GB, the canonical daily-driver laptop, and a Mac M5 Max—using a public MLX harness. Stacking 4-bit weights with KV quantization and prefill tuning reduces Llama 3.1 8B from 16.3 GB / 5.6 tokens/s to 5.1 GB / 19.9 tokens/s on the M3, enough headroom to keep the rest of the computer usable. Speculative decoding adds up to $1.8\times$ more decode throughput when draft acceptance is high, and fails when it is not. We close with recipes for 16/24/64+ GB machines, a symptom-to-lever decision procedure, a comparison to prior surveys, and open problems that remain once the cloud is no longer the default runtime.

1 Introduction

Transformer language models [95, 13] generate text one token at a time. Datacenter serving systems have spent three years making that loop fast for *many concurrent users* on HBM-rich GPUs [49, 106, 3]. A different, larger population of users has a different problem: they want a capable open-weight model to run on the computer they already own—a 16–32 GB laptop, a phone, a tablet—without sending every keystroke to an API, without a discrete GPU, and without closing the rest of their life (browser, IDE, mail, notes).

That setting is not a scaled-down datacenter. On everyday devices the CPU, GPU, and often an NPU share one DRAM pool [7, 8]. Weights, the KV cache, activations, the operating system, and Chrome compete for the same gigabytes. Decode is typically memory-bandwidth bound [99, 78]: every generated token re-reads almost the entire model. Prefill of a RAG prompt can stall the UI for tens of seconds even when streaming tokens/s looks fine. There is usually *one* interactive user, so

*Corresponding author. Update affiliation and email before arXiv upload.

continuous batching and tensor parallelism are the wrong first knobs. Thermals and battery replace rack power [64]. Privacy and offline availability are product requirements, not afterthoughts [12, 53, 33].

The selling point of this survey is therefore operational: **which inference optimizations make 7–9B (and smaller) models daily drivers on small, already-busy machines**—and which famous serving techniques can wait until the user actually has a cluster. We do not ignore the datacenter literature. We re-rank it.

Why now. Open weights in the 0.5B–9B band [23, 44, 43, 104, 29, 1, 107] crossed a quality threshold for coding assistants, summarization, and private RAG. Local runtimes—llama.cpp [31], MLX [8, 9], MLC LLM [71], ExecuTorch [67], Ollama [74]—made those checkpoints one command away. Apple Intelligence ships a ~3B on-device model as a consumer feature [33]. The remaining question is no longer “can it run?” It is “can it run *while the machine is still a computer*?”

Contributions.

1. A bottleneck-first taxonomy of *all* major inference optimization families, written so that stacking is an equation (memory floor from weights, slope from KV, TTFT from prefill, decode from bandwidth and speculation) rather than a list of papers.
2. A device-centric reading of the same literature: what matters on a contended 24 GB laptop or a phone versus a multi-GPU server, including related work on on-device LMs [103, 79], small LMs [63, 96], and cloud serving [68, 111].
3. A reproducible empirical slice on Apple Silicon (Mac M3 24 GB and Mac M5 Max): weight bits, KV quantization, prefill chunking, speculative decoding, context/RAG, model ladder, and two local runtimes.
4. Recipes and a symptom-to-lever map aimed at practitioners who *are* the ops team.

Scope. Inference-time methods: post-training compression, caches, kernels, decoding algorithms, serving schedules, parallelism, runtimes, and prompt-side controls. Training-time pretraining recipes are out of scope except where the same primitive (tensor/pipeline/expert parallel, distillation, LoRA) is reused at serve time. We mention phones, NPUs, and PCs; the measurements in Section 16 are Apple Silicon laptops, which we treat as the highest-volume “serious local LLM” platform today, not as the only one.

Organization. Section 2 is related work. Section 3 states the everyday-device constraint. Section 4 reviews the pipeline and memory budget. Section 5 gives the taxonomy. Sections 6–15 treat each family. Section 16 reports measurements. Sections 17–19 give recipes, open problems, and conclusions.

2 Related work

A survey of inference optimizations has to say what is already surveyed, what is missing, and where this paper sits. We group prior work into five clusters.

2.1 Surveys of efficient and serving-oriented inference

Miao et al. [68] review generative LLM serving from algorithms to systems, with emphasis on batching, parallelism, and datacenter engines (Orca, vLLM, FasterTransformer, DeepSpeed-Inference). Zhou et al. [111] catalog efficient inference algorithms—quantization, pruning, distillation, speculative decoding, sparse attention—at a breadth that this paper inherits. Both are essential. Both treat the *device* as an accelerator with private high-bandwidth memory and the *workload* as many concurrent requests. Everyday machines invert both assumptions: memory is unified and contended, and concurrency is usually one human. Techniques that dominate those surveys (continuous batching, PagedAttention under load, tensor parallel across NVLink) are real, and we cover them, but they are not the first lever on a 24 GB MacBook.

2.2 On-device, edge, and small language models

A second wave of surveys takes the device seriously. Xu et al. [103] review on-device language models, covering compression, collaborative edge–cloud deployment, and vendor case studies. Qu et al. [79] survey edge LLMs from design through execution and applications. Lu et al. [63] and Wang et al. [96] survey *small* language models (roughly 10^8 – 5×10^9 parameters), including on-device latency and memory measurements. Personal LLM agents [53] discuss capability, efficiency, and security of assistants that live next to user data. TinyML surveys [103] address microcontrollers; that regime is below the 0.5B–9B models this paper measures, but the motivation—latency, privacy, no WAN—is the same.

What those reviews still under-emphasize is *stacking* of serving-style knobs on a daily-driver laptop: weight bits $\times$ KV bits $\times$ prefill chunking $\times$ speculation $\times$ prompt length, with a harness that isolates OOMs, and with an explicit contrast to cloud serving. That is the empirical gap we fill.

2.3 Model families designed to fit a pocket or a laptop

Architecture work has moved the Pareto frontier of quality versus size. Phi-3 [1] targets “highly capable locally on your phone.” TinyLlama [107], OpenELM [66], and MobileLLM [60] optimize sub-billion to few-billion transformers for edge NPUs. Llama 3.2 1B/3B, Gemma 2 2B [29], and Qwen 2.5 0.5B–3B [80, 104] made “a useful model in 1 GB” a default Hugging Face download. Mixtral [44] and DeepSeek-style MoE [58, 19] change the *active* FLOP budget. Apple’s foundation models [33] show that a $\sim$ 3B on-device model plus a larger server model is a shipping product, not a research sketch. Scaling laws [46, 36, 97] still argue that bigger is better in the limit; everyday devices force a *local* optimum: the largest model that remains interactive *and* leaves RAM for the OS.

2.4 Compression, caches, kernels, and decoding (primary papers)

Weight quantization for neural nets is classical [42, 34]. LLM-scale post-training methods include GPTQ [27], AWQ [56], LLM.int8() [20], SmoothQuant [100], OmniQuant [85], SpQR [22], SqueezeLLM [47], QuIP# [94], AQLM [24], QuaRot [10], and SpinQuant [59]. QServe [57] co-designs W4A8KV4 with a serving stack. Pruning [26, 65] and distillation [35, 81] produce different checkpoints rather than runtime flags.

KV-cache work includes GQA/MQA [86, 4], PagedAttention [49], KIVI [62], KVQuant [38], H₂O [108], StreamingLLM [102], SnapKV [54], Quest [93], and DuoAttention [101]. Attention kernels are FlashAttention 1/2/3 [18, 17, 84] and online softmax [70]; long-prefill acceleration includes MInference [45] and chunked prefill [2]. Linear-time sequence models [32] are an architectural alternative.

Speculative and parallel decoding include Leviathan et al. [51], Chen et al. [15], Medusa [14], EA-GLUE [55], Lookahead [28], blockwise parallel decoding [91], Jacobi-style methods [82], staged speculation [90], and SpecInfer [69]. Contextual sparsity at decode [61] is a related skip-compute idea.

Serving systems: Orca [106], vLLM [49], Sarathi-Serve [3], Splitwise [75], DistServe [110], SGLang [109], DeepSpeed-Inference [6], TensorRT-LLM [73], TGI [41]. Parallelism: Megatron [88, 72], GPipe [40], GShard / Switch / Mixtral [50, 25, 44], EdgeMoE [105]. Offload: FlexGen [87], LLM in a flash [5].

2.5 Local runtimes and consumer-GPU systems

llama.cpp and GGUF [31, 30] made CPU/Metal/CUDA inference a weekend project. MLX [8, 9] is Apple’s array framework with Metal kernels and a first-class `mlx-1m` generate path. MLC LLM [71] compiles models via TVM for many backends. ExecuTorch [67] targets mobile. Ollama [74] wraps llama.cpp-class engines for distribution. PowerInfer [89] exploits activation locality to run large models on a *consumer* GPU plus CPU. These systems are the reason the rest of this paper is not hypothetical: the optimizations are already flags in software people install.

Table 1: How this survey relates to representative prior reviews. “Device-first” means the default reader is a single-user laptop/phone, not a GPU cluster.

Work	Device-first	Measurements on daily laptops	Emphasis
Miao et al. [68]	No	No	Serving systems, batching, parallelism
Zhou et al. [111]	Partial	No	Broad algorithm catalog
Xu et al. [103]	Yes	Mixed / review	On-device LMs, compression, edge-cloud
Qu et al. [79]	Yes	Review	Edge design, execution, applications
Lu et al. [63]	Yes	Phone/IoT-scale SLMs	Small models, capability vs. cost
Wang et al. [96]	Yes	Review	SLM architectures and training
Li et al. [53]	Yes	No	Personal agents, security
This paper	Yes	Yes (M3 24 GB, M5 Max)	Full optimization stack + stacking math

2.6 Where we differ

Table 1 summarizes positioning. We keep the algorithm catalog of Zhou et al. [111] and the serving map of Miao et al. [68], adopt the on-device motivation of Xu et al. [103], Lu et al. [63], and add (i) a single stacked memory/throughput model, (ii) an explicit re-ranking of techniques for *one user*, *shared DRAM*, *background apps*, and (iii) end-to-end measurements on a 24 GB daily-driver Mac and a workstation-class M-series chip.

3 Everyday devices as the inference platform

3.1 What we mean by “everyday”

We mean machines that are already someone’s computer, not a rented A100:

- **Thin laptops** (16–24 GB unified or shared memory): MacBook Air/Pro M-series, Windows ARM/x86 notebooks, Chromebooks with Linux containers.
- **Workstations that are still desks** (32–128 GB): Mac Studio / M-series Max/Ultra, a single consumer NVIDIA GPU (8–24 GB VRAM) plus host DRAM.
- **Phones and tablets** (6–16 GB): NPUs (Apple Neural Engine, Qualcomm Hexagon, Google Tensor), where a 1–3B 4-bit model is the product [1, 33, 60].

Raspberry Pi-class boards and microcontrollers sit below this band; datacenter racks sit above it. The interesting engineering is the middle: models that people *want* (7–9B instruct, sometimes 27–32B) on memory that people *have*.

3.2 Why local inference is the product

Four reasons dominate, and none of them is “benchmarks look nice.”

Privacy and data gravity. Source code, mail, medical notes, and unpublished documents should not have to leave the machine for a coding assistant to work [12, 53]. On-device inference is the only architecture that makes “the model never sees the network” a default rather than a contractual clause.

Latency without a WAN. TTFT includes the round trip to an API. On a train, a plane, a hospital basement, or a flaky guest Wi-Fi, that round trip is the product. Local prefill is slower in FLOPS but has no tail from congestion.

Cost at daily volume. A power user generating millions of tokens per month against a frontier API pays a metered bill. A 4-bit 8B model on a laptop they already bought has near-zero marginal token cost. Energy is not free [64, 76], but it is amortized into a device they recharge nightly.

Agency and reproducibility. Local checkpoints pin the model. Prompts, adapters, and RAG corpora stay inspectable. That matters for science, for regulated workflows, and for people who do not want a silent prompt-library change on Tuesday.

3.3 The constraint that is not in the GPU papers

On a discrete GPU, you fill VRAM first; host DRAM is a different pool; the OS is someone else’s problem. On unified-memory SoCs [7] (Figure 2), **there is one ceiling**. An 8B FP16 checkpoint (≈ 16 GB of weights, Eq. (2)) on a 24 GB Mac leaves roughly a browser tab’s worth of slack before swap. Swap on a laptop is not “slow inference.” It is a hung UI.

Consumer NVIDIA laptops with 8 GB VRAM hit a dual-pool version of the same wall: the model must fit in VRAM *or* pay PCIe for every layer (FlexGen-style offload [87], PowerInfer-style hot/cold split [89], LLM-in-a-flash [5]). Phones are stricter: a 3B 4-bit model plus OS plus camera pipeline is the budget [1, 33].

The optimization agenda follows: shrink the *floor* (weights), insure the *slope* (KV as T grows), keep TTFT interactive (prefill and prompt hygiene), then spend leftover RAM on speculation or a second model. Datacenter knobs—page the KV of *hundreds* of sequences, shard a 70B across eight GPUs—are the other end of the same catalog.

3.4 Metrics that match daily use

We track peak resident memory (will Activity Monitor go red?), TTFT (did the app freeze after Enter?), and decode tokens/s (does the stream feel like typing?). Serving papers add p99 under load and goodput [3]; those matter the day the laptop becomes a home server. Quality is a constraint: 2-bit that wins a tokens/s chart and fails a code-review is not a daily driver. Energy and skin temperature are first-class on phones and fanless laptops; they are still under-reported (Section 18).

4 Background: the inference pipeline

4.1 Prefill and decode

Let a request have prompt length T_p and generation length T_g , with $T = T_p + T_g$. Inference has two phases (Figure 1):

1. **Prefill.** The model consumes all T_p prompt tokens, writes the KV cache, and emits the first generated token. The user-visible metric is time-to-first-token (TTFT).
2. **Decode.** The model emits tokens $2, \dots, T_g$ autoregressively. The user-visible metric is decode throughput (tokens/s). Each step attends over a cache of length $T_p + t$.

Confusing the two phases is the most common optimization error on laptops. A change that triples tokens/s can leave a 15 s RAG pause untouched. A Flash-style kernel can collapse TTFT on long prompts while barely moving streaming speed. Product dashboards that only plot tokens/s ship demos that look fine in CI and feel broken on real prompts.

4.2 Memory budget

Peak resident memory is

$$M_{\text{peak}} \approx M_{\text{weights}} + M_{\text{KV}} + M_{\text{act}} + M_{\text{overhead}}, \quad (1)$$

with static weights

$$M_{\text{weights}} \approx \frac{N \cdot b_w}{8} \quad (2)$$

Autoregressive Inference Pipeline (What We Measure)

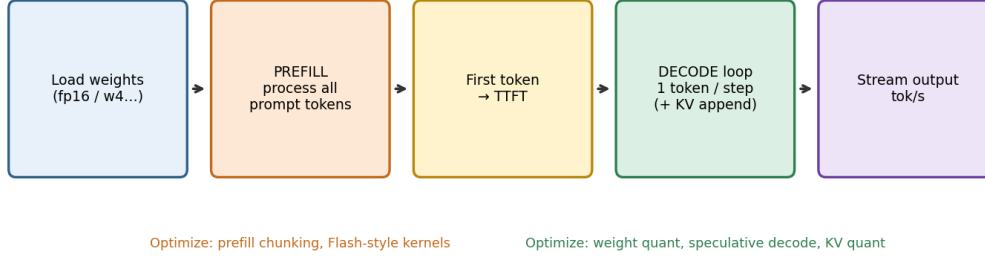


Figure 1: Single-request inference pipeline: load weights, prefill the prompt (TTFT), then autoregressive decode (tokens/s). Peak memory is a property of the whole run—the quantity everyday devices actually run out of.

for N parameters at b_w bits, and a KV cache

$$M_{KV} \approx 2 L H_{kv} T D \frac{b_{kv}}{8} \quad (3)$$

for L layers, H_{kv} key/value heads, head dimension D , sequence length T , and b_{kv} bits per cached element [78, 4]. The factor of two is the pair (K, V) . Grouped-query and multi-query attention reduce H_{kv} relative to query heads [86, 4]. M_{overhead} includes the framework, allocator fragmentation, and—critically on everyday devices—whatever the OS and apps already hold.

For a Llama-class 8B model, $b_w = 16$ costs ≈ 16 GB of weights. At $b_w = 4$ the same term is ≈ 4 GB. For $L = 32$, $H_{kv} = 8$, $D = 128$, $T = 640$, FP16 KV is ≈ 84 MB; 4-bit KV is ≈ 21 MB. At $T = 4096$, FP16 KV is ≈ 536 MB. Weights set the *floor*; KV sets the *slope* in T . Short-chat benches are weight-dominated and will lie about KV optimizations (Section 7).

4.3 Roofline view of decode

When a decode step is memory-bandwidth bound [99],

$$\text{tok/s} \approx \frac{B_{\text{mem}}}{B_{\text{token}}}, \quad (4)$$

where B_{mem} is sustained DRAM (or HBM) bandwidth and B_{token} is bytes moved per generated token—dominated by reading the weights. Reducing b_w therefore raises tokens/s *and* lowers M_{weights} . On a laptop this is why 4-bit is not only a fit trick: Llama 3.1 8B on an M3 moves from 5.8 to 20.5 tokens/s when $b_w : 16 \rightarrow 4$, close to the byte-ratio prediction after kernel overheads.

Prefill can be much more compute- or attention-IO-bound as T_p grows. The same 4-bit checkpoint that streams well can still post a 15 s TTFT at $T_p = 2048$.

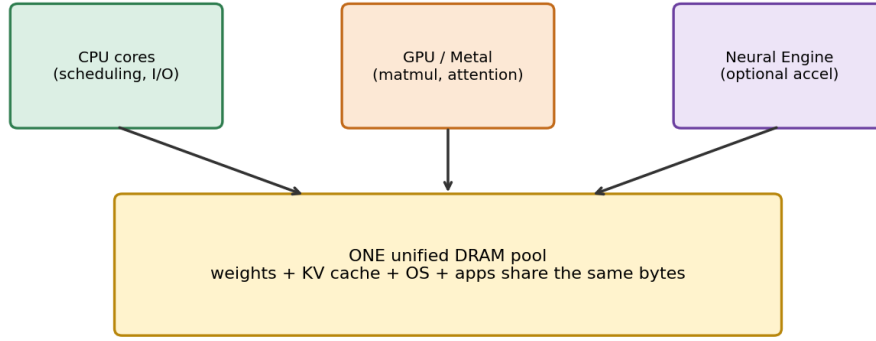
4.4 Unified memory versus discrete GPUs

On a discrete GPU, weights live in HBM; the host CPU has a separate DRAM pool; PCIe is an explicit copy path. On Apple Silicon, CPU, GPU, and Neural Engine share one unified DRAM pool [7, 8] (Figure 2). There is no host–device copy in the CUDA sense, and also *no private VRAM ceiling*. Local 8B inference on a 24 GB Mac is a packing problem first and a kernel problem second.

4.5 Quality as a constraint, not a metric to maximize in this paper

We do not claim that 4-bit matches FP16 on every reasoning bench. GPTQ/AWQ-class 4-bit is widely treated as the practical default for local chat and coding [27, 56]; 2-bit and naive 3-bit remain

Unified Memory on Apple Silicon — Why Quantization Matters



Discrete GPU PC: separate VRAM · Mac: total RAM is the hard ceiling

Figure 2: Unified memory on Apple Silicon: weights, KV cache, activations, and the operating system share one DRAM pool. Capacity planning is the first optimization on everyday devices.

Table 2: Taxonomy of LLM inference optimizations. “Daily-driver rank” is how early a solo user on a 16–24 GB machine should reach for the family.

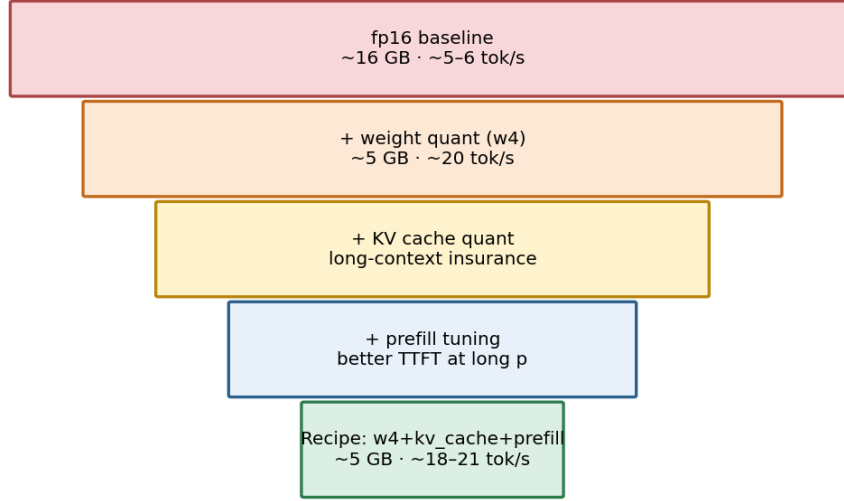
Family	Representative methods	Bottleneck	Daily rank
Weight / activation compression	FP16/BF16, INT8/4/2, GPTQ, AWQ, SmoothQuant, QuaRot, pruning, distillation	Capacity + decode bandwidth	1st
Architecture & size	0.5B–9B ladder, Phi/OpenELM/MobileLLM, MoE, long-context variants	Quality vs. fit	1st
KV cache & context	KV quant, GQA/MQA, PagedAttention, prefix cache, H2O, StreamingLLM, SnapKV	Capacity slope in T	2nd
Attention & prefill	FlashAttention, chunked prefill, MInference, sparse/local attn, RoPE/YaRN	Prefill compute / IO	3rd
Decode algorithms	Speculative decoding, Medusa, EAGLE, SpecInfer, graphs	Sequential decode	4th
Compilation & runtimes	MLX, llama.cpp, MLC, ExecuTorch, compile, of-fload, mmap	Kernel efficiency	4th
Application-level	RAG sizing, prompt cache, constrained decoding, early exit	Tokens billed / prefilled	always
Batching & scheduling	Continuous batching, SLOs, disaggregated PD	Multi-request utilization	late
Parallelism	Tensor, pipeline, expert, data parallel	Multi-device fit/latency	late
Adapters	LoRA / QLoRA, multi-LoRA	Extra matmuls + RAM	as needed

research-to-product. A responsible local stack uses 4-bit as the daily driver and keeps a W8/FP16 checkpoint for eval nights (Section 17).

5 A bottleneck-first taxonomy

Table 2 organizes techniques by the bottleneck they attack. Families are complementary: weight bits do not replace KV paging; speculative decoding does not shrink the checkpoint. On everyday devices the *order* is not optional.

Stacking Optimizations — Funnel from Slow/Heavy → Fast/Lean



Optional next lever: speculative decoding (draft model) → Part 7

Figure 3: Local optimization funnel: weight bits shrink the floor, KV quantization insures growth in T , prefill tuning attacks TTFT, and the daily recipe is the combination that survives the RAM budget *with the rest of the computer still running*.

A useful stacking order for *single-stream* local inference is: (1) choose a model that can fit; (2) quantize weights until there is OS headroom; (3) quantize or page KV if context grows; (4) tune prefill and *shorten the prompt* if TTFT hurts; (5) add speculation if decode still lags and RAM remains; (6) only then change runtime or hardware. Multi-tenant serving inverts the last steps: paging and continuous batching become mandatory before exotic decode algorithms. Figure 3 sketches the local funnel.

6 Weight and activation compression

Weight compression is the first optimization on everyday devices because it attacks both terms that make a laptop unusable: the memory *floor* (2) and the bytes moved per decode step (4).

6.1 Affine quantization

Affine quantization [42] maps a real weight x to a b -bit integer code

$$q = \text{clip}(\text{round}(\frac{x}{s} + z), 0, 2^b - 1), \quad \hat{x} = s(q - z), \quad (5)$$

with scale s and zero-point z , typically shared by a group of G weights (e.g. 64 or 128). Per-tensor scales are cheaper to store but fail when a matrix has a few huge outliers; per-channel or group-wise scales are the local default. Reconstruction error per element is on the order of half a quantization bin, $|x - \hat{x}| \lesssim s/2$. Lower b forces a larger typical s for the same dynamic range, which is the quality-size tradeoff.

A numeric sketch: $s = 0.05$, $z = 8$, $x = 0.42$ yields $q = \text{round}(0.42/0.05 + 8) = 16$ and $\hat{x} = 0.40$. Matmuls use many such $\hat{x}$, not the original FP16 values. Two 4-bit codes pack into one byte,

$$\text{byte} = (q_0 \bmod 16) + 16 \cdot (q_1 \bmod 16),$$

Table 3: Ideal weight memory ($\approx Nb_w/8$) before KV, activations, and OS. 4-bit 8B is the 24 GB daily-driver default; 4-bit 32B wants a 64 GB-class machine.

Model N	$b_w = 16$	$b_w = 8$	$b_w = 4$
3B	≈ 6 GB	3 GB	1.5 GB
8B	≈ 16 GB	8 GB	4 GB
32B	≈ 64 GB	32 GB	16 GB
70B	≈ 140 GB	70 GB	35 GB

so kernels unpack, dequant, and multiply in registers; Python never loops per weight. Scale metadata M_{scales} is usually $< 1\%$ of weight bytes, so

$$M_{\text{weights}} \approx \frac{N \cdot b_w}{8}$$

remains the planning equation. Table 3 is the number a 16/24/64 GB owner actually needs.

GPTQ was motivated by 175B-class models that could not fit on a single GPU [27]; the same math now makes 8B models comfortable on a laptop—an inversion of the original constraint that is the everyday-device story.

6.2 Post-training methods for LLMs

Local stacks almost never run the calibrator at serve time. They *load* a checkpoint produced by one of the following.

GPTQ [27] quantizes weights column-by-column. After rounding a column, it uses a Hessian-aware update so remaining weights compensate for the rounding error (optimal brain compression in one shot). It needs a small calibration set and is the ancestor of many GGUF/GPTQ files.

AWQ [56] does not treat all channels equally. It measures activation magnitudes, identifies *salient* weight channels, and rescales or protects them so 4-bit rounding does not destroy the directions the network actually uses. In practice AWQ 4-bit is often the most robust “download and go” format for chat models.

LLM.int8() [20] keeps a tiny set of outlier features in 16-bit during 8-bit matmul. It explains why naive INT8 of transformers collapses: a few channels dominate.

SmoothQuant [100] migrates quantization difficulty from activations into weights via a per-channel smoothing factor s_j , enabling W8A8. That path matters more when activations are quantized (prefill, large batch) than in single-stream FP16-activation decode on a laptop.

OmniQuant [85] learns quantization parameters. Rotation / incoherence methods—QuaRot [10], SpinQuant [59], QuIP# [94]—apply orthogonal transforms so weights look more Gaussian and 4-bit (even 2-bit) bins are better used. Additive residual codebooks (AQLM [24]) and sparse-quantized hybrids (SpQR [22], SqueezeLLM [47]) store a tiny high-precision outlier set plus a compressed dense remainder. QServe [57] co-designs W4A8KV4 with a serving engine.

GGUF [30] exposes a zoo of layouts (Q4_K_M, Q5_K_M, Q8_0, IQ2_XXS, ...) that mix group sizes and super-blocks. “4-bit” in MLX, GPTQ, AWQ, and Q4_K_M are *not* the same random variable (Section 14).

Bit width $b_w \in \{16, 8, 4, 2\}$ is a first-class serving knob. Empirically, **4-bit is the default sweet spot on unified-memory laptops**: 8-bit still spends too much DRAM for a daily driver; 2-bit frequently shows quality cliffs (Section 16). On a 24 GB Mac, changing $b_w : 16 \rightarrow 4$ is both a fit trick and a speed trick: Llama 3.1 8B decode goes from ≈ 5.8 to ≈ 20.5 tokens/s because each step reads $\approx 4\times$ fewer weight bytes.

6.3 Activation quantization, mixed precision, pruning, distillation

Activation quantization (W8A8, W4A8) matters when GEMM is compute-bound or when activation RAM dominates—more common in prefill and large-batch serving than in single-stream decode, where weights dominate the traffic. Mixed precision (AMP) keeps accumulators in FP32 while storing weights in FP16/BF16; it is the default, not an extra flag, in most GPU stacks.

Pruning [34, 26, 65] removes weights. Unstructured sparsity needs specialized kernels to beat dense 4-bit GEMM on a laptop; if the kernel still reads a dense 4-bit block, you paid quality for no bandwidth win. Structured (channel/head) pruning is easier to serve but harder to keep accurate. Knowledge distillation [35, 81] produces a smaller student: it is a *different model*, not a runtime switch, but it is often the highest-leverage “optimization” if a 1–3B student meets the product bar—the Phi/TinyLlama/OpenELM bet [1, 107, 66].

7 KV cache and context memory

7.1 Why the cache exists

Scaled dot-product attention [95] is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right) V. \quad (6)$$

Recomputing K, V for the entire history at every decode step would repeat $O(T)$ work per token. Caching per-layer K and V makes decode $O(1)$ cache appends plus attention against the stored history. Equation (3) then grows linearly in T .

Worked Llama-class 8B numbers ($L = 32, H_{kv} = 8, D = 128$) make the “KV does nothing” myth go away:

T	FP16 KV	4-bit KV	When you feel it
640 (default bench)	≈ 84 MB	≈ 21 MB	Hidden under 5 GB weights
4096 (short RAG)	≈ 536 MB	≈ 134 MB	Visible on 16 GB
32k (long context)	≈ 4.3 GB	≈ 1.1 GB	Rivals the 4-bit checkpoint

On a 24 GB machine with 4-bit 8B weights (≈ 5 GB), the cache is a rounding error at $T = 640$ and first-class at long RAG, multi-turn agents, or many concurrent sequences. Short-chat A/B tests that “prove KV quant does nothing” are measuring the wrong T . Our M3 w4 vs. w4+KV deltas at default T are $\approx -1.5\%$ tokens/s—insurance, not a short-chat speedup.

7.2 Fewer KV heads

Multi-query attention [86] shares one KV head across all query heads ($H_{kv} = 1$). Grouped-query attention [4] interpolates, with $H_{kv} \ll H_q$ (Llama 3 8B uses $H_q = 32, H_{kv} = 8$, a $4\times$ cache cut versus MHA). This is an *architectural* reduction of M_{KV} and of cache bandwidth. It explains why two 7B models can have very different long-context RAM slopes, and why Llama-3-class GQA is so friendly to laptops. Multi-head latent attention in DeepSeek-class models [58] compresses KV further by storing a latent, then up-projecting—another architectural lever, not a serving flag.

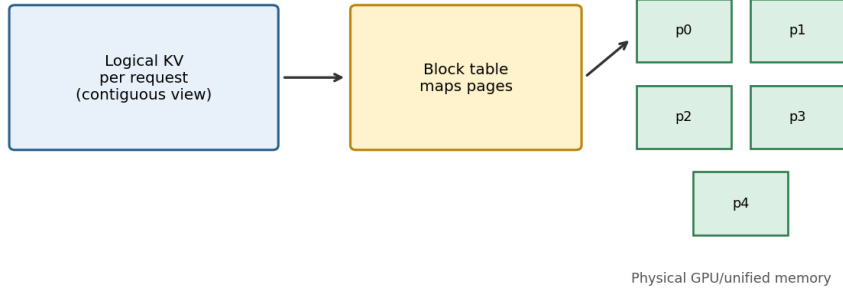
7.3 Quantizing the cache

KV quantization stores cached K, V at $b_{kv} < 16$ (commonly 8, 4, or 2). It is independent of b_w : FP16 weights + 4-bit KV is legal and useful when you refuse to quantize weights for an eval night but still want long T . Group-wise affine quantization of cache rows is the same algebra as weights. KIVI [62] uses *asymmetric* 2-bit KV (keys and values quantized differently) without fine-tuning. KVQuant [38] targets million-token contexts with per-channel and pre-RoPE tricks. Quality impact is usually smaller than aggressive weight quantization because each decode step only *reads* the cache rather than using it as the model’s entire knowledge. On-device, enable it as insurance once 4-bit weights have created headroom—especially on 16 GB machines.

7.4 Paging, prefix reuse, eviction, and sparsity

PagedAttention [49] stores KV in non-contiguous blocks of (typically) 16–64 tokens, analogous to virtual memory, so a serving engine can pack many sequences without reserving a giant contiguous buffer per request (Figure 4). A laptop running one chat does not need a pager; a laptop running

PagedAttention idea — KV in non-contiguous blocks (original redraw)



Cuts fragmentation for multi-request serving (vLLM). Local single-user MLX is simpler — same KV math.
Original redraw of the paged-KV idea from Kwon et al., Efficient Memory Management... / PagedAttention (2023)

Figure 4: Paged KV (redraw after Kwon et al., 2023). Multi-user serving pages the cache; a daily-driver laptop is the single-tenant cousin of the same memory problem.

local llama-server for a household does. Without paging, fragmentation from variable-length chats wastes the same DRAM the OS needs.

Prefix / prompt caching reuses KV for a repeated system prompt or RAG template of length T_{sys} . Cold TTFT pays prefill of $T_{\text{sys}} + T_{\text{user}}$; warm TTFT pays only T_{user} (plus a cache lookup). For a 2k-token system prompt and a 128-token question, that is the difference between “spinner” and “instant.” It is the local analogue of a provider’s prompt cache and one of the highest-ROI everyday-device tricks.

Eviction drops tokens believed unimportant: H₂O keeps heavy-hitter tokens with large attention mass [108]; StreamingLLM keeps an *attention sink* (first few tokens) plus a sliding window [102], which is why infinite-chat demos do not OOM; SnapKV [54] and Quest [93] select what to keep given the query; DuoAttention [101] splits retrieval heads from streaming heads. Sliding-window attention (Mistral-style) bounds KV by construction [43]: T in (3) is replaced by $\min(T, W)$. These trade exact long-range attention for a bounded RAM envelope—exactly the phone/laptop deal.

8 Attention kernels and prefill

Naive attention materializes $S \in \mathbb{R}^{n \times n}$ per head. At $n = 2048$ and FP16 that is already ≈ 8 MB per head per layer; times $L \times H$ this is gigabytes of intermediates—unacceptable on a laptop at long context. Compute is still $\Theta(n^2 d_h H)$ for the $QK^\top$ and softmax- V products.

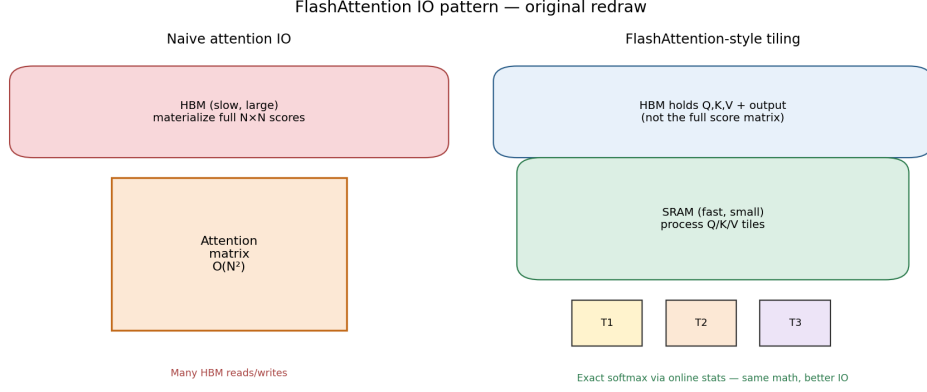
FlashAttention [18] tiles Q, K, V through on-chip SRAM, uses an online softmax [70], and never writes the full score matrix to HBM. For each block it tracks a running row-max m_i and row-sum ℓ_i :

$$m_i^{\text{new}} = \max(m_i^{\text{old}}, m_{ij}), \quad (7)$$

$$\ell_i^{\text{new}} = e^{m_i^{\text{old}} - m_i^{\text{new}}} \ell_i^{\text{old}} + e^{m_{ij} - m_i^{\text{new}}} \ell_{ij}, \quad (8)$$

and rescales the output accumulator so the result matches standard attention up to floating-point order. FLOPs remain $\Theta(n^2)$; extra memory is $O(n)$. FlashAttention-2/3 improve work partitioning and low-precision paths [17, 84]. On Apple Silicon, MLX implements tiled Metal attention internally; user code rarely toggles a “Flash” flag (Figure 5).

The user-visible leftover knob is often **chunked prefill** with step size C [2, 3]. A prompt of length T_p is processed in $\lceil T_p/C \rceil$ chunks. In our harness, “prefill off” is $C = 512$ and “on” is $C = 2048$. For $T_p = 512$ both are a single chunk (so the flag looks useless). For $T_p = 2048$, off means four chunks and on means one: less loop overhead and better kernel occupancy. Smaller C lowers peak



Original redraw of the tiled-attention IO idea from Dao et al., FlashAttention (2022) / FlashAttention-2 (2023)

Figure 5: IO-aware tiled attention (redraw after Dao et al.). Long RAG prompts on a laptop are exactly when avoiding the $n \times n$ score matrix matters.

activation memory and, in servers, lets decode tokens mix with partial prefills. Larger C often lowers TTFT when the device is under-occupied.

A simple TTFT model is

$$\text{TTFT} \approx \frac{\text{FLOPs}_{\text{prefill}}}{\text{GPU throughput}} + \frac{M_{\text{read}}}{B_{\text{mem}}}. \quad (9)$$

When attention dominates, doubling T_p can push TTFT up superlinearly. On M3 with Llama 3.1 8B w4 we measured ≈ 2.4 s at $T_p = 256$, ≈ 5.8 s at 1024, and ≈ 15.4 s at 2048 (Section 16).

Long-prompt prefill is where everyday RAG dies. MInference [45] sparsifies prefill attention dynamically. Sparse and local attention [16, 11] reduce the $O(n^2)$ term by restricting the pattern. State-space models [32] change the architecture to linear time. Rotary embeddings [92] and YaRN [77] extend usable context without a full pretrain; they do not by themselves reduce M_{KV} .

On a MacBook, the first prefill optimization is still application-level: retrieve fewer chunks (Section 15). The second is prefix cache. The third is kernels and C .

9 Decode-time algorithms

9.1 Speculative decoding

Baseline decode costs $\approx T_g \cdot \tau_t$ where τ_t is one target-model forward, so tok/s $\approx 1/\tau_t$. Speculative decoding [51, 15] uses a cheap draft to propose k tokens; the target verifies them in one parallel forward and accepts a prefix of length $m \leq k$ under a rejection-sampling rule that preserves the target distribution (Figure 6). The first rejected position is resampled from the target; remaining draft tokens are discarded.

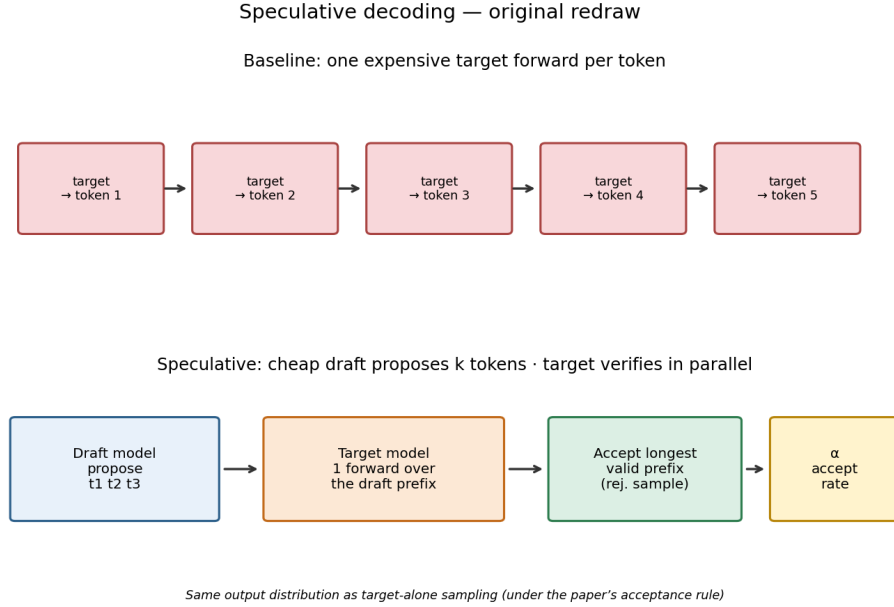
Let α be the per-token acceptance probability (in our harness, `draft_accept_rate`). Under an i.i.d. approximation the expected accepted length per round is $(1 - \alpha^{k+1})/(1 - \alpha) - 1$ plus the bonus resampled token. Idealized wall-clock speedup when verification batches well is

$$\text{speedup} \approx \frac{k \cdot \tau_t}{\tau_d + \tau_{\text{verify}}}, \quad (10)$$

and in practice we see $1.4\text{--}2\times$ when $\alpha \gtrsim 0.7$ and $\tau_d \ll \tau_t$. If the draft is a poor match, one still pays $\tau_d + \tau_{\text{verify}}$ and can *lose* throughput—we measure this on Llama 3.1 8B at $\alpha = 58.6\%$ (Section 16).

Memory is the every-day-device constraint:

$$M_{\text{peak}} \approx M_{\text{target}} + M_{\text{draft}} + M_{KV,t} + M_{KV,d}. \quad (11)$$



Original redraw of the draft/verify idea from Leviathan et al. (2023) and speculative sampling (Chen et al., 2023)

Figure 6: Speculative decoding (redraw after Leviathan et al. and Chen et al.). Speedup tracks acceptance α and draft cost, not merely the presence of a draft—critical on memory-tight laptops.

A 0.5B 4-bit draft is ≈ 0.25 GB—cheap only after 4-bit *target* weights have freed ~ 11 GB relative to FP16. Draft and target must share tokenizer vocabulary. Our MLX path uses `num_draft_tokens = k = 3` and a family-matched small instruct model.

Medusa [14] attaches extra heads to the target instead of a second model—attractive on phones where loading two graphs hurts. **EAGLE** [55] drafts at the feature level using a lightweight autoregressive head on hidden states. **SpecInfer** [69] verifies token *trees*. Lookahead and blockwise methods [28, 91, 90] and Jacobi-style parallel decoding [82] iterate several future positions. Deja Vu [61] skips compute via contextual sparsity rather than drafting tokens.

On everyday devices speculation is a *fourth* lever: it needs leftover RAM, a compatible draft, and a kill-switch when α drops. CUDA graphs / Metal capture replay a static decode graph and cut launch overhead once shapes stabilize; they help small-batch latency and are complementary, not a substitute for bits.

10 Batching, scheduling, and disaggregation

Local `generate()` loops are single-stream. Production serving is not. We include this family because (i) the same human may later run `llama-server` for a household, and (ii) papers in this family invented paging and chunked prefill that local stacks now expose as flags.

Static batching waits to fill a batch of size B , pads to the longest sequence, and holds every request until the slowest finishes. It under-utilizes bursty interactive traffic and inflates latency.

Continuous batching (iteration-level scheduling) [106, 49] mixes prefill of new requests with decode of in-flight ones at every model forward. The unit of scheduling is a step, not a whole request. Combined with PagedAttention, this is why vLLM can keep a GPU full without reserving worst-case KV per session.

Chunked prefill in Sarathi-Serve [3] piggybacks decode tokens alongside partial prefills so a 32k-token prompt cannot stall every interactive user. The same idea appears as `prefill_step_size` on a laptop, where the “other users” are just the rest of your decode stream.

Disaggregated prefill/decode [75, 110] places the two phases on different pools because prefill is compute-heavy and decode is KV-/bandwidth-heavy; the objective is goodput, not raw tokens/s. A household analogue would be “prefill on the desk Mac, decode on the laptop,” which almost nobody runs yet (Section 18).

SGLang [109] adds a radix cache (prefix tree over KV) and compiled structured generation. Request scheduling adds SLOs, preemption, and priority (chat versus batch jobs). These policies do not change Equations (1)–(4); they change *who* holds the KV pages and *when* a prefill is admitted.

On a daily-driver laptop with one user, continuous batching does almost nothing. On a Mac mini serving a family, it suddenly does. The mistake is cargo-culting vLLM flags before the model fits beside Safari.

11 Parallelism

When one device cannot hold the model, inference reuses training sharding.

Tensor parallelism [88, 72] splits GEMMs inside a layer (column-parallel W_{up} , row-parallel W_{down}) and all-reduces activations. Latency can drop if interconnect is fast (NVLink); it *rises* if all-reduce dominates a small decode step—the usual case at batch 1 on consumer PCIe.

Pipeline parallelism [40] assigns consecutive layers to devices. Bubble time is painful at batch 1 unless micro-batches exist; a single interactive user is the worst case.

Expert parallelism for MoE [25, 50, 44] routes tokens to expert shards. Memory is proportional to *total* experts; compute per token is proportional to *active* experts. Mixtral $8 \times 7\text{B}$ is a 47B-class memory object with $\approx 13\text{B}$ active parameters: attractive on 64 GB, painful on 24 GB. EdgeMoE [105] specifically targets on-device MoE by prefetching experts.

Data parallelism replicates weights and splits the batch; natural for large-batch throughput, useless for one chat.

Consumer Macs almost never expose multi-GPU tensor parallel for LLMs; the relevant decision is “does this dense 70B fit in unified memory at 4-bit?” (usually no on 24 GB; $\approx 35\text{ GB}$ of weights plus KV on 128 GB Ultra is plausible). A single consumer NVIDIA GPU plus CPU/SSD offload [87, 89, 5] is the PC analogue: PowerInfer keeps hot neurons on GPU and cold ones on CPU; LLM-in-a-flash pages weights from NAND. These are *fit* strategies that sacrifice tokens/s. They are how a gamer GPU runs a 70B at all, not how a 24 GB Mac should serve an 8B.

12 Architecture and model-size choices

The highest-leverage “optimization” on a small machine is sometimes a smaller model. Memory (2) and bandwidth-bound decode (4) both scale with N . A dense 0.5B 4-bit model and a dense 8B 4-bit model can differ by an order of magnitude in tokens/s on the same laptop (Section 16): Qwen 2.5 0.5B at 238 tokens/s versus Llama 3.1 8B at 21 tokens/s versus Gemma 2 9B at 15 tokens/s, all at 4-bit on M3.

Phi-3 [1], TinyLlama [107], OpenELM [66], MobileLLM [60], Gemma 2 2B [29], and Llama 3.2 1B/3B exist because of that curve: they are not “toy models,” they are the models that fit beside iOS and Android. Mixture-of-experts [44, 58] offers more total parameters at a smaller active-FLOP budget, at the cost of a larger memory floor. Long-context variants (32k–128k) do not change N but inflate T in (3) and prefill cost; buying a 128k checkpoint without KV quant or sliding windows is how 24 GB machines swap. Instruction-tuned versus base checkpoints should be held fixed in systems benchmarks; they are not speed knobs.

A practical ladder for everyday devices: 0.5B–1B for always-on / phone; 3B for a capable pocket model [33, 1]; 7–9B 4-bit for a 24 GB laptop assistant; 27–32B 4-bit for 64 GB desks; 70B only with quantization, offload, or a workstation.

Table 4: Local inference engines a daily-driver user actually installs. Match quant *tier* and T_p, T_g , not marketing names.

Engine	What it is	Everyday-device role
MLX / mlx-lm [8, 9]	Metal-native arrays, <code>mlx-community</code> checkpoints, Python <code>stream_generate</code>	This paper’s harness; best when you already live in Python on a Mac
llama.cpp / GGUF [31, 30]	C/C++, Metal/CUDA/CPU, Q4_K_M and friends	Default portable engine; CPU fallback when GPU RAM is gone
Ollama [74]	UX and model library over llama.cpp-class engines	Distribution, not a third algorithm family
MLC LLM [71]	TVM compilation across GPU/NPU/CPU	Phones and heterogeneous NPUs
ExecuTorch [67]	On-device PyTorch	Android/iOS productization
PowerInfer [89]	Hot-neuron GPU + cold-neuron CPU	Consumer NVIDIA PC running models that do not fit in VRAM

13 Adapters

LoRA [39] injects low-rank updates $\Delta W = BA$ with $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$. Parameter count per adapted matrix is $r(d + k)$ instead of dk . QLoRA [21] fine-tunes those adapters on a quantized base—the reason a laptop can specialize an 8B model overnight without a second 16 GB copy of the weights. At inference, each adapter adds a small extra GEMM (xA then $(\cdot)B$) and a small RAM footprint (often tens of megabytes per adapter at $r = 16$). Multi-LoRA serving keeps many adapters resident and batches across different B, A pairs; that is a server problem (vLLM, SGLang). For a single local user, one LoRA is usually noise compared with b_w , and it is how “my coding style” becomes an on-device file rather than a cloud fine-tune.

14 Compilation, kernels, and runtimes

Graph compilation (`torch.compile`, `mlx.compile`, XLA, TensorRT-LLM [73], MLC [71]) fuses ops, specializes shapes, and can capture a decode step as a single graph. Quantized matmul kernels determine whether 4-bit is actually faster than FP16 or merely smaller: naive dequant-then-FP16-GEMM can lose the roofline win by doubling traffic. Fused dequant-GEMM (Marlin-class CUDA kernels, MLX Metal INT4, llama.cpp q4_K SIMD/Metal) is the difference between “4-bit is slower” folklore and the $3.5\times$ we measure.

CPU/disk offload [87, 5] runs models that do not fit, at a large bandwidth penalty (PCIe or NAND vs. DRAM). Memory-mapping weights from SSD reduces cold-start RAM spikes: the OS loads pages on first touch. That helps *launch*, not steady-state decode, unless you accept thrash.

Local runtimes illustrate that *the same bit width is not the same system* (Table 4):

Fair comparison requires matched prompts, generation length, and a comparable quant tier (FP16 vs. F16 GGUF; 4-bit MLX vs. Q4_K_M). Decode metric in llama.cpp `llama-bench` is `tg` tokens/s; prefill is `pp` tokens/s, which is *not* TTFT. Our Article 10 JSON reports both and a ratio $R = \text{tok}/s_{\text{MLX}}/\text{tg}_{\text{llama.cpp}}$. Neither engine universally wins; kernel quality and thread pinning dominate.

15 Application-level inference

Not every latency problem belongs inside the model—especially on everyday devices, where the user controls the prompt.

Table 5: Llama 3.1 8B weight-bit sweep on Mac M3 (24 GB), $T_p = 512$, $T_g = 128$.

Config	Peak GB	TTFT (ms)	tok/s	vs fp16
fp16	16.33	2637	5.8	1.0×
w8	8.96	2775	11.3	1.9×
w4	5.06	2738	20.5	3.5×
w2	3.11	2826	35.8	6.2×

Retrieval-augmented generation [52] can *increase* TTFT by stuffing thousands of tokens into T_p . If a retriever returns k chunks of c tokens, prefill cost scales with kc (and attention with $(kc)^2$). Cutting k in half is often a larger TTFT win than a new kernel. Rerank-then-truncate, smaller chunk size, and “answer from memory unless retrieval is needed” are inference optimizations. Prefix caches absorb repeated templates. Semantic prompt caches reuse prior completions for identical questions (agents that retry tools).

Constrained / structured decoding [98, 109] masks logits toward JSON or grammars; it adds CPU work per token and can reduce sample diversity, but it prevents the “retry the whole 8B forward because the JSON was invalid” loop—often a net win on a laptop. Early-exit and confidence stopping [83] cut T_g when the model is already sure. Nucleus and temperature [37] are quality knobs with mild speed effects via average T_g . Tool/function calling [48] is an application graph; on a laptop, serial tool loops (search $\rightarrow$ read $\rightarrow$ generate) dominate wall-clock more than GEMM.

The daily-driver heuristic: **every token you do not retrieve is a token you do not prefill.**

16 Empirical study: daily-driver laptops

The taxonomy is only useful if the local, measurable subset is actually measured on the class of machine people use. We summarize a public harness² that holds prompt length, generation length, and trial policy fixed, and varies one family at a time and then in combination.

Hardware. Mac M3 with **24 GB** unified memory—the canonical daily-driver laptop in this study—and Mac M5 Max (workstation-class M-series). Same prompts, same token counts, same config labels.

Default shape. $T_p = 512$, $T_g = 128$, unless noted. Models are mlx-community checkpoints (Llama 3.1, Mistral, Qwen, Gemma, and related presets). Configurations: fp16, w8, w4, w2, with optional +kv_cache (4-bit KV) and +prefill (prefill_step_size = 2048 vs. 512).

16.1 Weight quantization dominates the floor

On Mac M3, Llama 3.1 8B (Table 5):

FP16 at 16.3 GB on a 24 GB machine is not a daily driver: the OS and a browser barely fit. 4-bit is the knee of the memory–speed Pareto (Figure 7): the laptop stays a laptop. 2-bit wins the leaderboard and is the first place quality complaints concentrate; we treat it as a last resort, not a silent default. Across 14 M3-feasible presets the same pattern holds: fewer bits raise tokens/s because decode is bandwidth-bound (4).

16.2 Stacking KV quantization and prefill

Table 6 is the capstone. Adding 4-bit KV and larger prefill chunks on top of 4-bit weights yields **5.06 GB** and **19.9 tokens/s** for Llama 3.1 8B on M3 ($\approx 3.55\times$ decode vs. FP16, $\approx 31\%$ of FP16 memory). Mistral 7B moves from 3.6 to 16.0 tokens/s and 14.8 to 4.6 GB ($\approx 4.4\times$). The ≈ 11 GB clawed back from Llama FP16 is enough to hold another 7–8B 4-bit model, or a fat RAG cache, *or Chrome*.

²Companion repository: LLM-Inference. Isolated subprocesses, schema-versioned JSON, median of trials after one warmup.

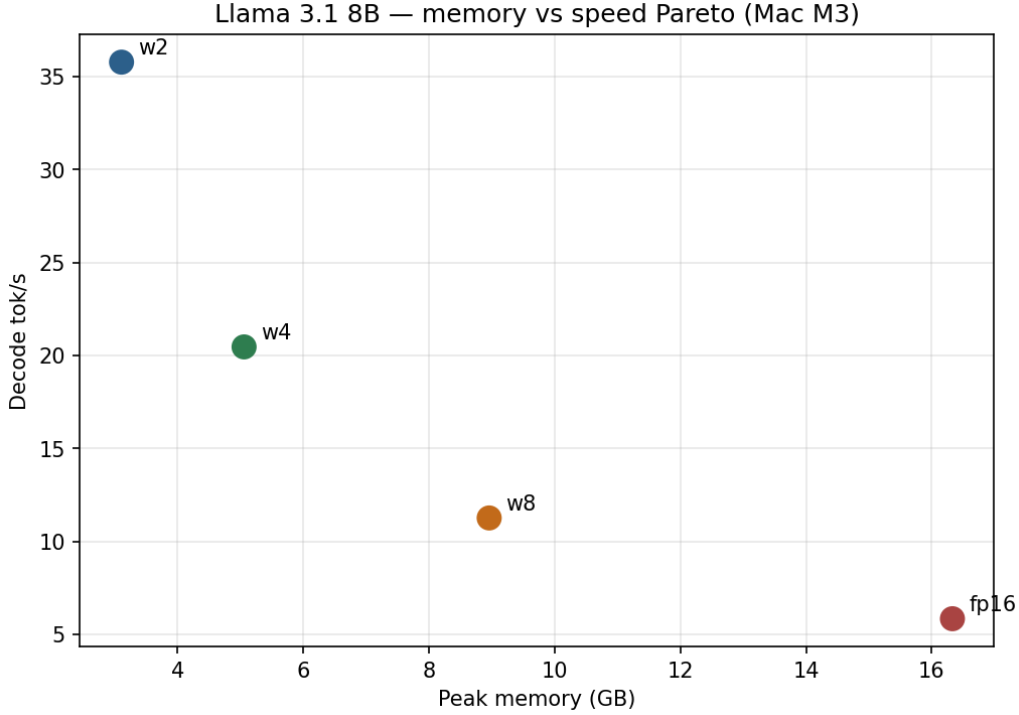


Figure 7: Memory–speed Pareto across weight bit-widths and models. Moving from FP16 to 4-bit is a joint win on both axes for bandwidth-bound decode—the difference between “science fair” and “I leave this running all day.”

Table 6: Full stack on Mac M3 (24 GB). Fully optimized = w4+kv_cache+prefill.

Model / recipe	Peak GB	TTFT (ms)	tok/s
Llama 3.1 8B, fp16	16.33	2689	5.6
Llama 3.1 8B, full stack	5.06	2746	19.9
Mistral 7B, fp16	14.77	4350	3.6
Mistral 7B, full stack	4.62	3954	16.0

TTFT barely moves on this *short* prompt. That is expected: prefill’s value shows up when T_p climbs. KV quantization’s absolute gigabyte win is small at $T = 640$; it is insurance for long T . On short chats, enabling KV+prefill on top of w4 can even cost a few tokens/s. Ship w4+kv_cache+prefill as a *product preset*; bench plain w4 if you want peak short-chat speed.

At default T , Llama 3.1 8B w4 vs. w4+KV on M3 is 20.7 vs. 20.4 tokens/s (about -1.5%); Mistral and Qwen 2.5 7B similarly. The knob is not a dud—the benchmark is weight-dominated.

16.3 The 16-config matrix on M5 Max

Weight bits $\times$ KV on/off $\times$ prefill on/off is $4 \times 2 \times 2 = 16$ configs. Table 7 (Llama 3.1 8B, M5 Max) shows the ladder: fp16 ≈ 35 tok/s at 16.5 GB, w8 ≈ 64 at 9.0 GB, w4 ≈ 112 at 5.1 GB, w2 ≈ 178 at 3.3 GB. KV and prefill barely change short-prompt decode. Mistral 7B on the same machine: fp16 ≈ 37 tok/s at 14.9 GB $\rightarrow$ optimized ≈ 114 tok/s at 4.7 GB.

Relative stacking math is portable across M3 and M5. Absolute UX is not. Memory geometry is portable: ≈ 5 GB for an 8B w4 stack is the planning number on both. M5 makes “FP16 for quality nights” realistic (≈ 35 tok/s); on M3, FP16 8B is a last resort.

Table 7: Selected cells of the Llama 3.1 8B matrix on Mac M5 Max (decode tok/s / peak GB).

Config	tok/s	Peak GB
fp16	35.1	16.46
w8	63.6	8.97
w4	112.5	5.11
w4+kv+prefill	≈ 107	5.11
w2	178.0	3.26

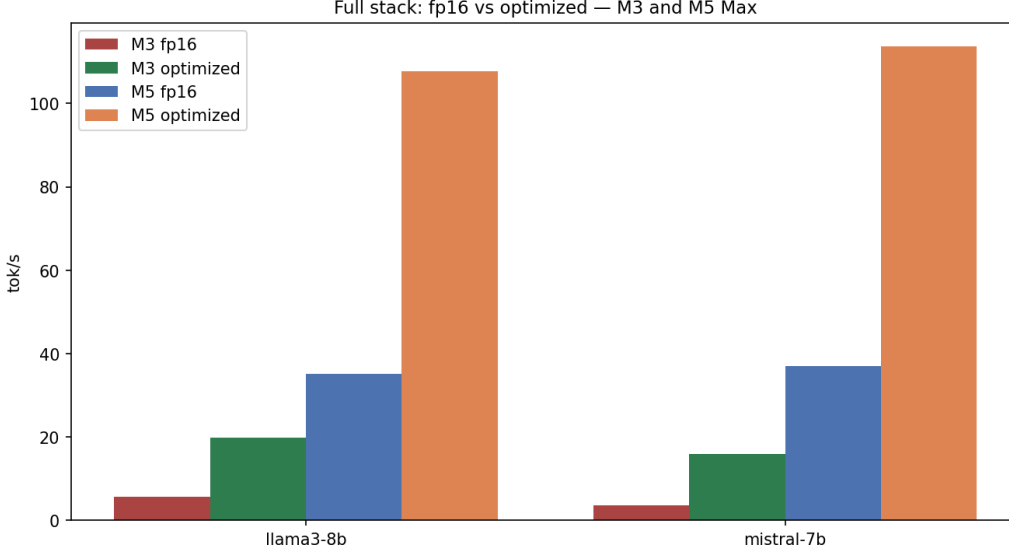


Figure 8: Same software stack, two chips. Relative wins from leaving FP16 travel; absolute UX does not: ≈ 20 tok/s on M3 is usable chat, $\approx 100+$ tok/s on M5 Max makes the UI the bottleneck.

16.4 Model-size ladder

Holding the recipe at 4-bit, M3 decode spans **238 tokens/s** at Qwen 2.5 0.5B (0.64 GB) down to **20.6 tokens/s** at Llama 3.1 8B (5.06 GB) and **15.4 tokens/s** at Gemma 2 9B (5.88 GB)—about $15\times$ from the tiny tier to the top of the “real laptop” tier before leaving single-digit billions of parameters (Figure 9). On M5 Max, Llama 8B 4-bit reaches ≈ 112 tokens/s and Gemma 27B 4-bit remains interactive at ≈ 33 tokens/s. Hardware bandwidth multiplies the same stack; it does not replace quantization on a 24 GB machine. If you pick models by social-media vibes alone you will either swap a 70B to death or run a 0.5B and wonder why reasoning collapsed.

16.5 Speculative decoding is not unconditionally faster

With a small 4-bit draft, Qwen 2.5 7B on M3 goes from 15.9 to **28.3 tokens/s** ($1.78\times$) at acceptance $\alpha = 74.2\%$, adding only ≈ 0.3 GB—possible only because 4-bit weights already freed RAM. The same pair on M5 Max goes from 122 to 170.3 tokens/s at the same α . Llama 3.1 8B on M5 Max is a negative result: $113.5 \rightarrow 109.7$ tokens/s at $\alpha = 58.6\%$. Speculation should be reported with α and a baseline, never as a guaranteed badge on a laptop preset.

16.6 Context, RAG, and prefix cache

Prefill TTFT grows steeply with T_p (Figure 10). On M3 with Llama 3.1 8B w4: $T_p = 256 \rightarrow 2.4$ s TTFT; $T_p = 512 \rightarrow \approx 3.1$ s; $T_p = 1024 \rightarrow 5.8$ s; $T_p = 2048 \rightarrow \approx 15.4$ s. A rag_agent workload with $T_p = 4096$ records TTFT ≈ 31.1 s on M3 versus ≈ 1.5 s on M5 Max. That is a product failure on the smaller chip even when decode tokens/s look acceptable. Prefix KV cache converts a repeated system prompt from a prefill into a cache hit. Generation-length sweeps confirm

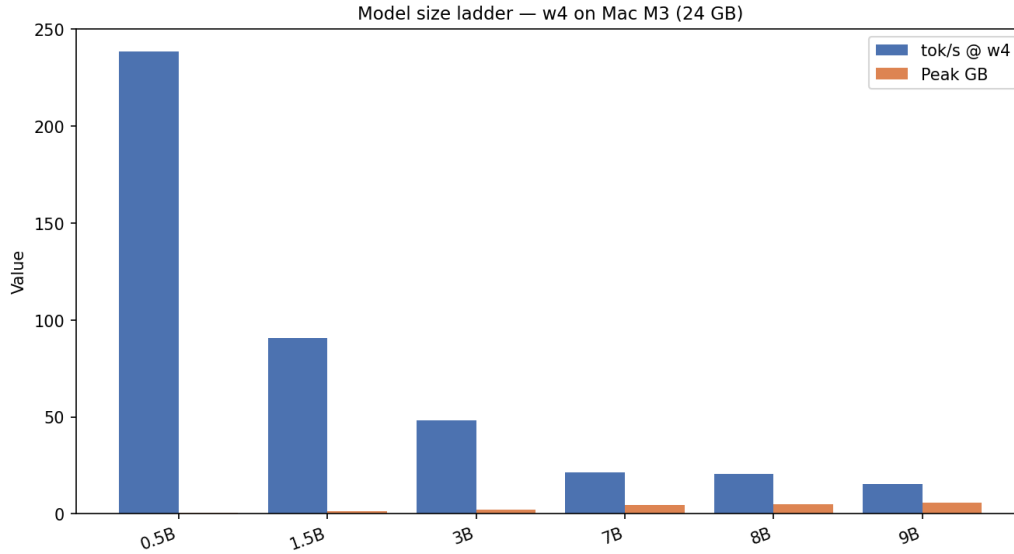


Figure 9: Dense 4-bit model ladder on Apple Silicon. Size is an inference optimization: quality, memory, and tokens/s move together.

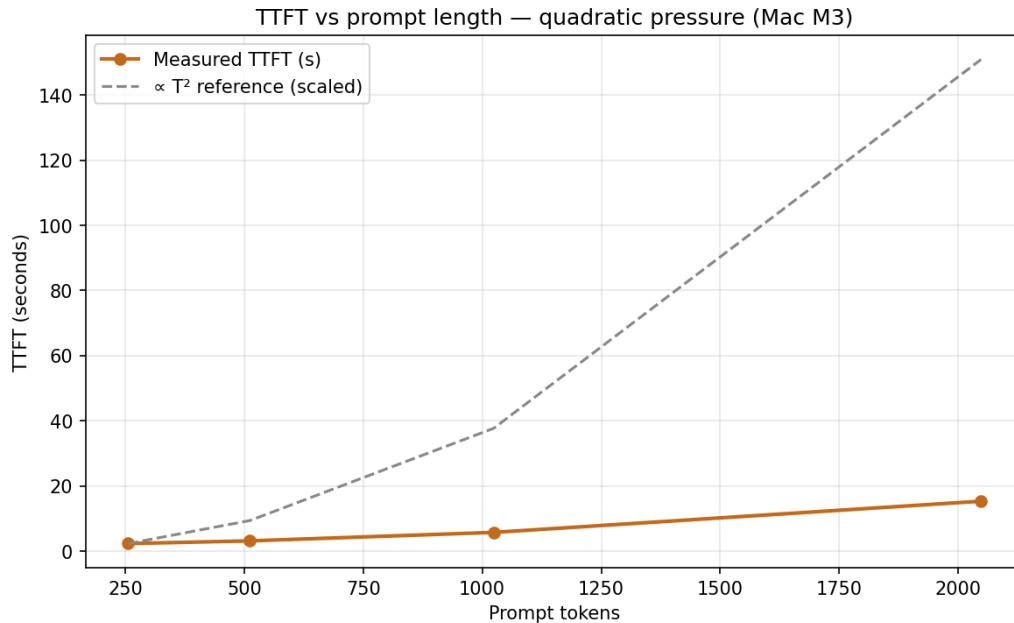


Figure 10: TTFT versus prompt length. Local RAG dies in prefill, not in the tokens/s column. Everyday-device UX is a prefill SLO.

Equation (3): decode tokens/s decay slowly as attention over the cache grows; peak memory tracks T .

16.7 Runtimes

MLX and llama.cpp implement overlapping quant tiers on the same Metal GPU. In matched-tier comparisons (FP16 vs. F16 GGUF; 4-bit vs. Q4_K_M), neither engine universally wins; kernel quality, thread pinning, and quant layout dominate. Ollama is a packaging layer. The practical rule:

Table 8: Symptom $\rightarrow$ first optimization on an everyday device. Later rows assume earlier ones are already sane.

Symptom	First lever
OOM / swap / hung UI	Smaller N , then $b_w \rightarrow 4$
Fits but slow stream	b_w , then faster SoC / DRAM
Fine on chat, dies on PDF/RAG	Shorter T_p , KV quant, prefix cache
Spinner before first token	Reduce T_p ; prefill kernels / chunks
Slow stream, RAM left over	Speculative decoding if α is high
Fan / battery / thermals	Smaller N or b_w ; cap threads
Many concurrent users	Paged KV + continuous batching
Model larger than one device	Offload, TP/PP, or do not run it locally
Need a specialist	One LoRA on a frozen 4-bit base
Same bits, disappointing speed	Runtime / kernel / compile
Quality regression after 4-bit	W8 or better quant; do not silent-W2

pick the runtime that exposes the lever you need (MLX for Python sweeps; llama.cpp for portable GGUF and CPU fallback), then freeze it while you sweep b_w and T .

17 Recipes and a decision procedure

17.1 Steal these presets

Daily driver — 24 GB laptop (M3-class). Config w4+kv_cache+prefill; models Llama 8B / Mistral 7B / Qwen 7B; expect ≈ 4.6 –5.1 GB peak and ≈ 16 –21 tokens/s. Do not use FP16 8B as the always-on default.

16 GB laptop or large phone. Stay at 1B–3B @ 4-bit as the assistant (Phi-3-class, Llama 3.2 3B, Qwen 2.5 3B); 7–8B @ 4-bit only with discipline and KV quant on. Skip FP16 7B+ entirely.

Quality / eval night, same 24 GB machine. W8, or FP16 with other apps closed. Better fidelity, half or full memory pain. Not the always-on assistant.

M5 Max / 64 GB+ desk. Chat default still w4+kv_cache+prefill (≈ 107 tok/s Llama 8B). Short-chat peak: plain w4 (≈ 112). FP16 8B is now an interactive eval mode (≈ 35 tok/s). 27B @ 4-bit is usable. 70B still wants quantization and a sober look at T .

Phone / NPU. Ship a 1–3B 4-bit (or vendor foundation model [33, 1]); treat 7B as optional on flagship RAM only; prefer Medusa-style heads over a second draft graph if memory is the wall.

17.2 Symptom $\rightarrow$ first lever

Table 8 maps the failure the user actually feels. Capacity before kernels, kernels before speculation, single-stream before multi-tenant.

18 Open problems

Quality under compression on real tasks. Systems papers still under-report tool use, coding, and long-context reasoning at 4- and 2-bit. A cheap eval suite that travels with every GGUF/MLX quant would change local practice more than another 5% kernel win.

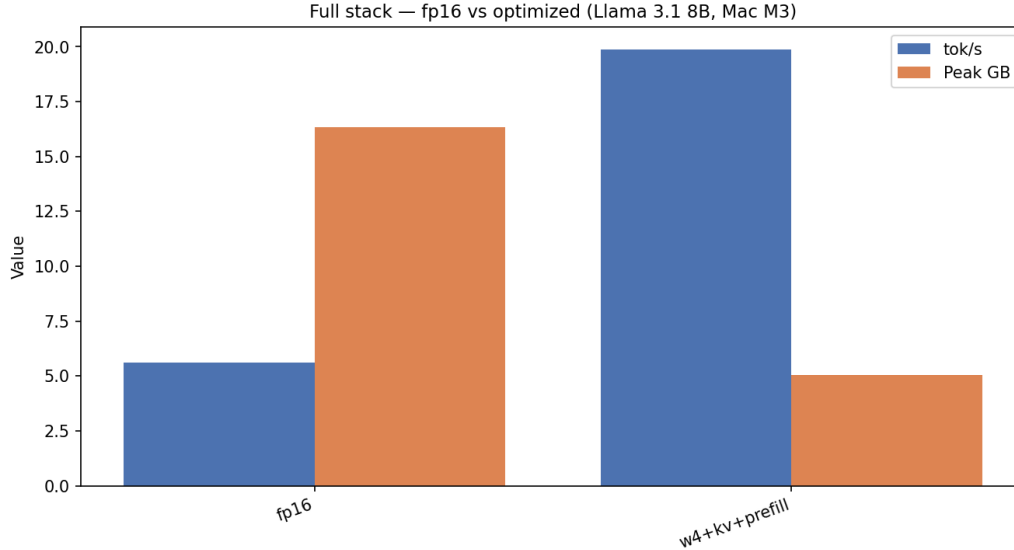


Figure 11: Mac M3, Llama 3.1 8B: FP16 versus the stacked 4-bit + KV + prefill recipe. Memory falls to $\approx 31\%$ of baseline; decode rises $\approx 3.55\times$ —enough that the machine remains a daily-use computer.

Contended unified memory as a first-class model. Roofline assumes a GPU with private HBM. Unified DRAM, OS interference, and background apps make B_{mem} a random variable. We need protocols that treat the laptop as a busy device, not an idle accelerator—including tests with a browser and IDE open.

When speculation fails. Acceptance α is a property of the draft–target pair, temperature, and domain. Adaptive k , draft routing, and “disable speculation when α drops” should be default runtime features, especially when RAM is barely enough for two models.

Prefill as a product SLO. Industry dashboards over-index on decode tokens/s. RAG and agents die on TTFT. Chunked prefill, prefix caches, retrieve-less application design, and MInference-style sparse prefill need the same status as weight bits.

Cross-runtime equivalence. “4-bit” in MLX, GPTQ, AWQ, and GGUF Q4_K_M are not the same random variable. A quant-tier ontology with shared perplexity probes would make runtime bake-offs honest.

Energy, thermals, and carbon of local tokens. Tokens/s on a MacBook is eventually a skin-temperature problem; on a phone it is a battery problem [64]. Joules per token belong in local inference reports. Local serving also shifts carbon from datacenter to wall wart [76]; that accounting is almost absent.

Phones, NPUs, and Windows ARM. Our measurements are Apple Silicon laptops. ExecuTorch / Hexagon / Neural Engine paths, thermal throttling, and 6–12 GB phone budgets are the next empirical slice. Vendor on-device models [33] are closed enough that independent stacking studies are hard.

Heterogeneous local clusters. A household Mac mini plus a phone plus a spare PC is not Dist-Serve, but disaggregated prefill/decode ideas [75] might still apply. Almost no open harness studies that topology.

19 Conclusion

LLM inference optimization is not one trick, and it is not only a datacenter sport. On everyday devices it is a packing problem (does the model fit *beside the operating system*?), a bandwidth problem (how many bytes per token?), a prefill problem (how long is the prompt you stuffed?), and only then a sequential-decode problem (can we verify several tokens at once?). The techniques in this survey attack those bottlenecks independently and can be stacked. The selling point is practical: a 24 GB laptop can run a 7–9B assistant as a daily driver if you take 4-bit weights as default, insure KV for long T , treat TTFT as the UX metric that RAG actually hits, and treat speculative decoding as a conditional bonus. The same catalog, read from the other end, is vLLM-style paging and disaggregated serving. We hope the related-work map, the equations, and the measurements make it obvious which end to open—and that “it runs on my machine” starts meaning the machine the reader already uses all day.

Acknowledgments and Disclosure of Funding

Benchmark numbers were collected with an open MLX harness on Apple Silicon. We thank the authors of the cited systems and the mlx-community and GGUF contributors who make comparable checkpoints public. Update this acknowledgment with funding and hardware donors before arXiv v1 if applicable.

References

- [1] Marah Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadalla, Hany Awadallah, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024.
- [2] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nagarajan Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. Sarathi: Efficient LLM inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369*, 2023.
- [3] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nagarajan Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation*, 2024.
- [4] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.
- [5] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, S. Kotsenko, et al. LLM in a flash: Efficient large language model inference with limited memory. *arXiv preprint arXiv:2312.11514*, 2024.
- [6] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, et al. DeepSpeed-inference: Enabling efficient inference of transformer models at unprecedented scale. *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022.
- [7] Apple Inc. Apple announces Mac transition to Apple silicon. <https://www.apple.com/newsroom/2020/11/apple-announces-mac-transition-to-apple-silicon/>, 2020.
- [8] Apple Inc. MLX: Efficient machine learning on Apple silicon. <https://github.com/ml-explore/mlx>, 2023.
- [9] Apple Inc. mlx-lm: LLM utilities for MLX. <https://github.com/ml-explore/mlx-lm>, 2023.

- [10] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Martin Jaggi, Dan Alistarh, Torsten Hoefer, and Tijmen Blankevoort. QuaRot: Outlier-free 4-Bit inference in rotated LLMs. *arXiv preprint arXiv:2404.00456*, 2024.
- [11] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [12] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.
- [13] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 2020.
- [14] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.
- [15] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau Driessche, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.
- [16] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.
- [17] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
- [18] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.
- [19] DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [20] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems*, 2022.
- [21] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized language models. *Advances in Neural Information Processing Systems*, 2023.
- [22] Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefer, and Dan Alistarh. SpQR: A sparse-quantized representation for near-lossless LLM weight compression. *arXiv preprint arXiv:2306.03078*, 2023.
- [23] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [24] Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. Extreme compression of large language models via additive quantization. *arXiv preprint arXiv:2401.06118*, 2024.
- [25] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- [26] Elias Frantar and Dan Alistarh. SparseGPT: Massive language models can be accurately pruned in one-shot. 2023.

- [27] Elias Frantar, Saleh Ashkboos, Torsten Hoeffler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations*, 2023.
- [28] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Accelerating large language model decoding with lookahead decoding. *arXiv preprint arXiv:2402.02057*, 2024.
- [29] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.
- [30] Georgi Gerganov. GGUF: GPT-generated unified format. <https://github.com/ggerganov/ggml/blob/master/docs/gguf.md>, 2023.
- [31] Georgi Gerganov and contributors. llama.cpp. <https://github.com/ggml-org/llama.cpp>, 2023.
- [32] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- [33] Tom Gunter, Zirui Wang, Chong Wang, Ruoming Pang, Aonan Narayanan, Aosong Zhang, Bowen Zhang, Chen Chen, et al. Apple Intelligence foundation language models. *arXiv preprint arXiv:2407.21075*, 2024.
- [34] Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *International Conference on Learning Representations*, 2016.
- [35] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- [36] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.
- [37] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. 2020.
- [38] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.
- [39] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *International Conference on Learning Representations*, 2022.
- [40] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, Hyoungho Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, et al. GPipe: Efficient training of giant neural networks using pipeline parallelism. *arXiv preprint arXiv:1811.06965*, 2018.
- [41] Hugging Face. Text Generation Inference. 2023.
- [42] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2704–2713, 2018.
- [43] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023.
- [44] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

- [45] Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir Abecassis, et al. MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. *arXiv preprint arXiv:2407.02490*, 2024.
- [46] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [47] Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W. Mahoney, and Kurt Keutzer. SqueezeLLM: Dense-and-sparse quantization. *arXiv preprint arXiv:2306.07629*, 2023.
- [48] Sehoon Kim, Suhong Moon, Rohan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. LLMCompiler: An LLM compiler for parallel function calling. *arXiv preprint arXiv:2312.04511*, 2024.
- [49] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.
- [50] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. *International Conference on Learning Representations*, 2021.
- [51] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- [52] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 2020.
- [53] Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Liu, et al. Personal LLM agents: Insights and survey about the capability, efficiency and security. *arXiv preprint arXiv:2401.05459*, 2024.
- [54] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.
- [55] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*, 2024.
- [56] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proceedings of Machine Learning and Systems*, 2024.
- [57] Yujun Lin, Haotian Tang, Shang Yang, Zhekai Zhang, Guangxuan Xiao, Chuang Gan, and Song Han. QServe: W4A8KV4 quantization and system co-design for efficient LLM serving. *arXiv preprint arXiv:2405.04532*, 2024.
- [58] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chengzhi Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, et al. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.
- [59] Zechun Liu, Changsheng Zhao, Igor Fedorov, et al. SpinQuant: LLM quantization with learned rotations. *arXiv preprint arXiv:2405.16406*, 2024.

- [60] Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yinyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Chandra, et al. MobileLLM: Optimizing sub-billion parameter language models for on-device use cases. 2024.
- [61] Zichang Liu, Jue Wang, Tri Dao, Tianyi Zhou, Binhang Yuan, Zhao Song, Anshumali Shrivastava, Ce Zhang, Yuandong Tian, Christopher Re, and Beidi Chen. Deja Vu: Contextual sparsity for efficient LLMs at inference time. 2023.
- [62] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. *arXiv preprint arXiv:2402.02750*, 2024.
- [63] Zhenyan Lu, Xiang Li, Dongqi Cai, Rong Yi, Fangming Liu, Xiwen Zhang, Nicholas D. Lane, and Mengwei Xu. Small language models: Survey, measurements, and insights. *arXiv preprint arXiv:2409.15790*, 2024.
- [64] Alexandra Sasha Luccioni, Yacine Jernite, and Emma Strubell. Power hungry processing: Watts driving the cost of AI deployment? *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency*, 2024.
- [65] Xinyin Ma, Gongfan Fang, and Xinchao Wang. LLM-Pruner: On the structural pruning of large language models. *Advances in Neural Information Processing Systems*, 2023.
- [66] Sachin Mehta, Mohammad Hossein Sekhavat, Qingqing Cao, Maxwell Horton, Yanzi Jin, Chenfan Sun, et al. OpenELM: An efficient language model family with open training and inference framework. *arXiv preprint arXiv:2404.14619*, 2024.
- [67] Meta. ExecuTorch: On-device AI across mobile and edge devices. <https://pytorch.org/executorch-overview>, 2024.
- [68] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Hongyi Jin, Tianqi Chen, and Zhihao Jia. Towards efficient generative large language model serving: A survey from algorithms to systems. *arXiv preprint arXiv:2312.15234*, 2023.
- [69] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, et al. SpecInfer: Accelerating large language model serving with tree-based speculative inference and verification. 2024.
- [70] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867*, 2018.
- [71] MLC team. MLC LLM: Universal LLM deployment engine with ML compilation. <https://llm.mlc.ai>, 2023.
- [72] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prithvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. Efficient large-scale language model training on GPU clusters using Megatron-LM. 2021.
- [73] NVIDIA. TensorRT-LLM. 2024.
- [74] Ollama. Ollama. <https://ollama.com>, 2023.
- [75] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture*, 2024.
- [76] David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluís-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350*, 2021.
- [77] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. *arXiv preprint arXiv:2309.00071*, 2023.

- [78] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, James Devlin, Jacob Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems*, 2023.
- [79] Hongyu Qu et al. A review on edge large language models: Design, execution, and applications. *ACM Computing Surveys*, 2025.
- [80] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [81] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.
- [82] Andrea Santilli, Alessandro Severino, Emanuele Postolache, Valentino Maiorca, Michele Mancusi, Riccardo Marin, and Emanuele Rodolà. Accelerating transformer inference for translation via parallel decoding. *arXiv preprint arXiv:2305.10427*, 2023.
- [83] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. *Advances in Neural Information Processing Systems*, 2022.
- [84] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. *arXiv preprint arXiv:2407.08608*, 2024.
- [85] Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng Xu, Lirui Zhao, Zhiqian Li, Kaipeng Yang, Peng Zhang, et al. OmniQuant: Omnidirectionally calibrated quantization for large language models. *arXiv preprint arXiv:2308.13137*, 2024.
- [86] Noam Shazeer. Fast transformer decoding: One write-head is all you need. 2019.
- [87] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- [88] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [89] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. PowerInfer: Fast large language model serving with a consumer-grade GPU. 2024.
- [90] Benjamin Spector and Christopher Re. Accelerating LLM inference with staged speculative decoding. *arXiv preprint arXiv:2308.04623*, 2023.
- [91] Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems*, 2018.
- [92] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- [93] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context LLM inference. *arXiv preprint arXiv:2406.10774*, 2024.
- [94] Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. QuIP#: Even better LLM quantization with Hadamard incoherence and lattice codebooks. *arXiv preprint arXiv:2402.04396*, 2024.
- [95] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.
- [96] Fali Wang, Zhiwei Zhang, Xianren Zhang, Zongyu Wu, Tzuhao Mo, Qihao Qiu, Tao Wang, et al. A survey of small language models. *arXiv preprint arXiv:2410.20011*, 2024.

- [97] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022.
- [98] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.
- [99] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [100] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- [101] Guangxuan Xiao, Jiaming Tang, Jingwei Lin, et al. DuoAttention: Efficient long-context LLM inference with retrieval and streaming heads. *arXiv preprint arXiv:2410.10819*, 2024.
- [102] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *International Conference on Learning Representations*, 2024.
- [103] Jiajun Xu, Zhiyuan Li, Wei Chen, Qiang Wang, Xin Gao, Qi Cai, and Ziyuan Ling. On-device language models: A comprehensive review. *arXiv preprint arXiv:2409.00088*, 2024.
- [104] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.
- [105] Rong Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. Edge-MoE: Fast on-device inference of MoE-based large language models. *arXiv preprint arXiv:2308.14352*, 2023.
- [106] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation*, 2022.
- [107] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. TinyLlama: An open-source small language model. *arXiv preprint arXiv:2401.02385*, 2024.
- [108] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. H₂O: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 2023.
- [109] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, et al. SGLang: Efficient execution of structured language model programs. *Advances in Neural Information Processing Systems*, 2024.
- [110] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Huang, Yibo Zhu, Xin Zhang, Xin Wang, Hao Zhang, Haibo Li, et al. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation*, 2024.
- [111] Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, Shiyao Li, Yuming Lou, Lun-ing Wang, Zhihang Yuan, Xiuhong Li, et al. A survey on efficient inference for large language models. *arXiv preprint arXiv:2404.14294*, 2024.

Table 9: Technique checklist (survey coverage).

Technique	Section	Typical status
FP16 / BF16 baseline	6	measured
INT8 / 4-bit / 2-bit, GPTQ, AWQ, SmoothQuant	6	measured / systems
OmniQuant, QuaRot, SpinQuant, QuIP#, AQLM, SpQR	6	systems
Activation quantization, QServe W4A8KV4	6	systems
Pruning, distillation	6	reference
KV cache quantization, KIVI, KVQuant	7	measured / systems
GQA / MQA	7	architecture
PagedAttention	7, 10	systems
Prefix / prompt KV cache	7	measured
H2O, StreamingLLM, SnapKV, Quest, DuoAttention	7	systems
FlashAttention / tiled attention	8	measured (proxy)
Prefill chunking, MInference	8	measured / systems
Sparse attention, RoPE, YaRN, Mamba	8	architecture
Speculative decoding, Medusa, EAGLE, SpecInfer	9	measured / systems
CUDA graphs / Metal capture, Deja Vu	9	systems
Continuous / static batching, Sarathi, DistServe, Splitwise	10	systems
Tensor / pipeline / expert parallel, EdgeMoE	11	systems
Model-size ladder, Phi/TinyLlama/OpenELM/MobileLLM, MoE	12	measured / reference
LoRA / QLoRA / multi-LoRA	13	systems
Graph compile, offload, mmap, FlexGen, LLM-in-a-flash	14	systems
MLX, llama.cpp, MLC, ExecuTorch, Ollama, PowerInfer	14	measured / systems
RAG sizing, constrained decoding, early exit, tools	15	reference

A Catalog checklist

Table 9 lists techniques covered in the body. Status tags refer to the companion Apple Silicon harness: *measured* means a sweep exists; *systems* means the method is standard in datacenter or local engines; *architecture* means it is baked into the checkpoint; *reference* means surveyed from the literature.

B Reproducing the empirical slice

On Apple Silicon, with Python 3.10+ and Hugging Face access:

```
git clone <repo> LLM-Inference && cd LLM-Inference
./scripts/setup_env.sh && source .venv/bin/activate
./scripts/hf_login.sh
./scripts/run_article.sh 1 "Mac M3" # weight quantization
./scripts/run_article.sh 5 "Mac M3" # full stack
./scripts/run_article.sh 6 "Mac M3" # speculative decoding
./scripts/run_article.sh 7 "Mac M3" # context / RAG / prefix cache
```

Each run writes schema-versioned JSON (median TTFT, tokens/s, peak GB). Tables in Section 16 are medians from that schema. Large-model rows require a 64 GB+ machine and `-include-large`.

C arXiv and NeurIPS usage notes

This source uses the official NeurIPS 2025 style with the `preprint` option. Authors are visible and the NeurIPS checklist is omitted. A broad on-device survey is a natural **arXiv** paper (`cs.LG`, cross-list `cs.CL`, `cs.DC`). NeurIPS main track is a poor fit unless the empirical protocol is framed as Datasets & Benchmarks and cut to the page limit. A workshop on efficient or on-device ML is the more realistic conference path.